

Matiks App

System Analysis & Solution Prototype

SDE Internship Exploration Project

Angelica Das

Contents

Introduction 1

System Flow & Learning Loop 3

Observations & Areas for Improvement 4

Proposed Improvements & Solutions 5

Prototype & Engineering Implementation 6

Solution: Rated vs Unrated System 7

Frontend & Backend Architecture 8

Game Modes & Matchmaking Design 9

Rating System 9

Game Logic 10

Key Engineering Decisions 11

Prototype Demonstration Video 12

System Flow & Learning Loop

How the platform routes a player from login to improvement

System Architecture

Frontend (React Native / Expo)

- 1) WebSocket Connection
- 2) Backend (Node.js and Express)
- 3) Matchmaking and Game Logic
- 4) Score Calculation Event Broadcast

Core Gameplay

- Real time 1v1 competitive gameplay
- Detailed post match insights accuracy, speed, trends
- Graph based performance visualization
- Practice mode with configurable difficulty
- Instant score updates & live opponent comparison
- Time pressure drives speed and accuracy
- Multiple game modes: Math, Memory, Puzzle, Logic

Learning Loop

Play

Result

Analysis

Practice

Replay

Observations & Areas for Improvement

Based on user responses and feedback

Rating VS pressure

The rating system creates a fun and competitive environment, but at higher ratings, the game requires intense focus and feels less casual since losing affects the rating.

Feature inconsistency

Practice settings in detailed analysis available in Logic/Math is missing in Memory/Puzzle once the game is over

A matiks for everyone

Right now, Matiks mainly caters to math oriented players, but expanding the memory section with more subsections could make the game more engaging for the non math biased pool of users .

Not playable in low network zones

The game is difficult to play in low-network areas or during train journeys, even though those are situations where users would likely want to play it the most.

Mid match network disconnect

If network goes bad mid match , “something went wrong” comes , sometimes its intentional from one end and can be frustrating for the user

Community driven feed engagement

Right now, the feed mainly contains auto generated posts celebrating streaks, but it could be made more engaging by allowing users to interact and share content themselves.

Proposed Improvements & Solutions

Coz, Matiks was never meant to feel like just another app

Rating VS pressure

Introduce Rated vs Unrated matchmaking so players can choose between competitive progression and stress-free casual gameplay without affecting rating.

Not playable in low network zones

Add an offline safe gameplay mode using cached question sets and locally simulated opponents to preserve gameplay during unstable connectivity.

Feature inconsistency

Extend practice and detailed analysis features across Memory and Puzzle sections to maintain a consistent post-game learning experience across all categories.

Mid match network disconnect

Use temporary adaptive bot substitution and local game state caching so gameplay can continue smoothly until the disconnected player reconnects.

A matiks for everyone

Expand memory focused subsections and introduce broader puzzle categories such as Sudoku from the Daily section to create a more inclusive experience for users with strengths beyond mathematics.

Community driven feed engagement

Allow short user generated interactions alongside existing auto generated streak and challenge sharing, while limiting post length and adding moderation flags to maintain a focused, puzzle oriented feed experience.

PROTOTYPE & ENGINEERING IMPLEMENTATION

PROBLEM STATEMENT :

MATIKS uses a rating based progression system that makes gameplay engaging and competitive. However, as player ratings increase, many users become more cautious about experimenting or casually exploring unfamiliar puzzle categories because losses affect their ranking. For example, a player confident in KenKen may hesitate to try Cross Math despite being interested in it, since weaker performance could reduce their rating.

SOLUTION

What if the player had a say in if they want the pressure or not ?

- Introduced a Rated vs Unrated gameplay system
- Rated mode : Competitive play with Elo based ranking
- Unrated mode : Pressure free gameplay for practice and exploration
- Enables users to balance competition and learning and enjoyment

FRONTEND & BACKEND ARCHITECTURE

Frontend Architecture

- React Native (Expo) - cross platform mobile app
- Screens: Login , Home , Matchmaking , Game , ResultScreen
- Custom keypad input system for fast answer entry
- Real time state updates via Socket.IO client

Backend Architecture

- Node.js + Express - RESTful API and socket server
- Socket.IO - persistent WebSocket connections
- Matchmaking queues - in memory, mode aware
- Game state management - per session, event driven

Game Modes

Fastest Finger

- Shared question shown to both players
- First correct answer wins the point
- Tests reaction speed and accuracy
- High intensity, competitive format

Sprint Mode

- Each player gets independent questions
- Score as many as possible in time limit
- Tests sustained accuracy under pressure
- Consistent scoring ,no racing mechanic

Matchmaking Design

- Rated & Unrated queues are strictly separated
- No cross queue matching ,rating correctness preserved
- Bot fallback triggers when no opponent found in time
- Eliminates waiting , keeps players in the loop

Rating System

- ELO algorithm with K factor = 32
- Applied exclusively in Rated matches
- Zero sum model , prevents rating inflation
- Sub ratings per game category (planned)

Game Logic

- Procedural question generation for dynamic gameplay
- Supports +, −, ×, ÷ operations
- Division constrained to integer only results
- Questions generated in realtime during matches
- Separate logic for different game modes
- Server side answer validation
- Instant score synchronization between players
- Backend controlled match timers
- Realtime gameplay updates using Socket.IO
- Independent game state for every room
- Event driven multiplayer architecture
- Low latency multiplayer synchronization
- Automatic question progression after correct answers
- Lightweight game state handling for smoother gameplay
- Bot fallback system with simulated human like accuracy

Key Engineering Decisions

WebSockets over REST

Realtime bidirectional synchronization with low latency

Zero sum ELO

Prevents rating inflation and preserves rank integrity

Bot fallback

Reduces waiting time and improves engagement

Room based multiplayer architecture

Isolated state for concurrent matches

Backend controlled timers

Prevents timer desynchronization between players

Event driven socket architecture

Responsive realtime multiplayer interactions

Separate rated / unrated queues

Balances competition and casual gameplay

Mode based matchmaking

Fair pairing based on gameplay context

Integer constrained division

Avoids ambiguous decimal-based answers

Lightweight game state payloads

Faster synchronization and smoother gameplay

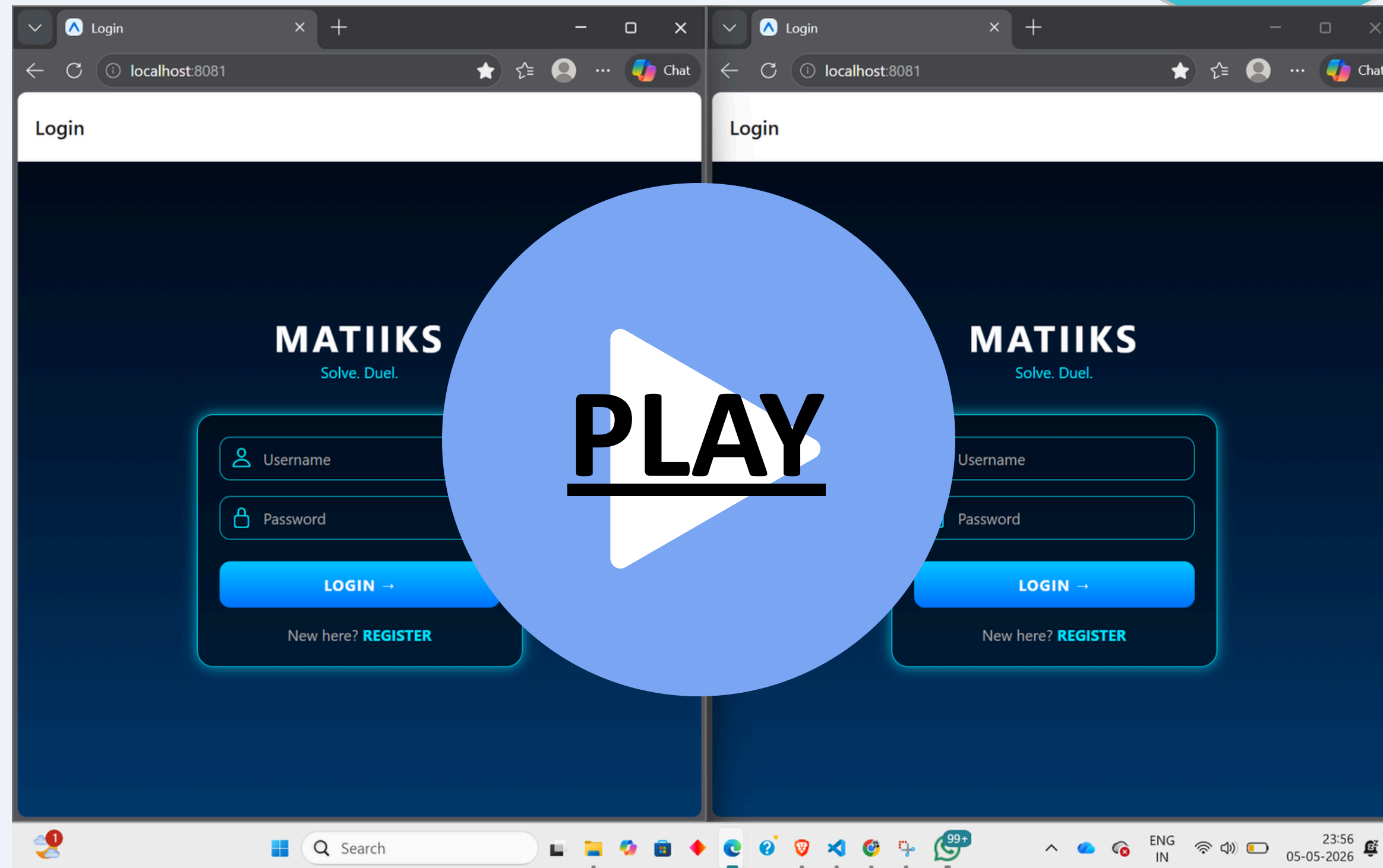
Procedural question generation

Dynamic gameplay across matches

Server authoritative validation

Prevents client-side manipulation and invalid scoring

VIDEO OF THE PROTOTYPE



Github Link: <https://github.com/lennisa/matiiks-prototype>